PYTHON FOR DATA SCIENCE · LESSON 26 OF 29

DSA I: Big-O, Arrays, Strings, Hashing, Two Pointers and Sliding Window

Data structures and algorithms for data-analyst, data-scientist and data-engineer interviews. Learn Big-O time and space complexity, then the patterns that solve most array and string questions (hash maps and sets, prefix sums, two pointers, sliding window), with worked interview problems and timings.

DSA for data roles (added) · Python 3.13 · Intermediate · 120 min

By the end of this lesson you can

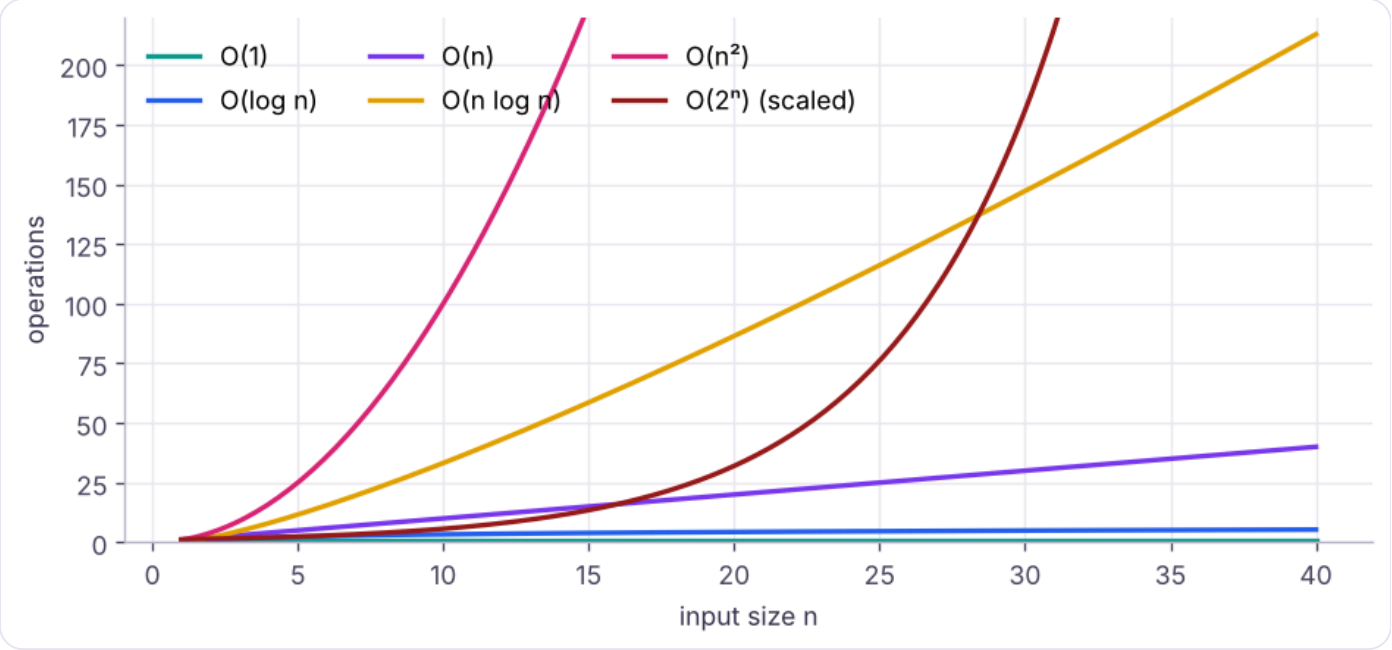
- Estimate time and space complexity with Big-O and compare common growth rates
- Know the cost of operations on lists, dicts, sets and deques
- Solve array and string problems with hash maps, counting and prefix sums
- Apply the two-pointer and sliding-window patterns
- Explain trade-offs out loud as in a coding interview

Why DSA for data roles?

Data interviews (Amazon, Flipkart, Walmart, fintechs, product companies) often include one or two coding rounds with easy–medium problems on arrays, strings, hash maps and SQL. Beyond interviews, complexity thinking explains why a join on a set is instant while a nested loop over two million rows never finishes.

Big-O notation

Big-O describes how running time (or memory) grows with input size *n*, ignoring constants.



Growth of common complexity classes. At n = 1,000,000 the difference between O(n log n) and O(n²) is days versus a second.

Big-O	Name	Example
O(1)	Constant	Dict lookup, list index, append
O(log n)	Logarithmic	Binary search, balanced-tree lookup
O(n)	Linear	One loop, `x in list`, `sum`
O(n log n)	Linearithmic	Sorting (`sorted`, Timsort), merge sort
O(n²)	Quadratic	Nested loops over the same data
O(2ⁿ)	Exponential	All subsets, naive recursion for Fibonacci

Rules: drop constants (O(2n) → O(n)); keep the dominant term (O(n² + n) → O(n²)); sequential steps add, nested steps multiply.

Python

```
import time
def has_duplicate_quadratic(a):
    for i in range(len(a)):
        for j in range(i + 1, len(a)):
            if a[i] == a[j]: return True
    return False
def has_duplicate_set(a): return len(set(a)) != len(a)
for n in (1000, 2000, 4000):
    data = list(range(n))
    t0 = time.perf_counter(); has_duplicate_quadratic(data); t1 = time.perf_counter();
has_duplicate_set(data); t2 = time.perf_counter()
    print(f"n={n:5d}: nested loops {1000*(t1-t0):7.1f} ms | set {1000*(t2-t1):.3f} ms")
```

OUTPUT

```
n= 1000: nested loops    15.1 ms | set 0.022 ms
n= 2000: nested loops    59.7 ms | set 0.042 ms
n= 4000: nested loops   261.3 ms | set 0.088 ms
```

Doubling *n* roughly quadruples the nested-loop time ($O(n^2)$) but only doubles the set version ($O(n)$).

Cost of Python operations

Operation	list	dict / set	deque
Index `a[i]`	O(1)	—	O(n)
Append / pop at end	O(1)	add O(1)	O(1)
Insert / pop at front	**O(n)**	—	**O(1)**
`x in c`	**O(n)**	**O(1)**	O(n)
Sort	O(n log n)	—	—

Pattern 1: hash maps and counting

Turn "search inside a loop" ($O(n^2)$) into "look up in a dict/set" ($O(n)$).

Python

```python
def two_sum(nums, target):
    """Return indices of two numbers adding up to target. O(n) time, O(n) space."""
    seen = {}                                   # value → index
    for i, x in enumerate(nums):
        if target - x in seen: return [seen[target - x], i]
        seen[x] = i
    return []
print(two_sum([2, 7, 11, 15], 9), two_sum([3, 2, 4], 6), two_sum([1, 2], 7))

from collections import Counter, defaultdict
def is_anagram(a, b): return Counter(a) == Counter(b)
def group_anagrams(words):
    g = defaultdict(list)
    for w in words: g["".join(sorted(w))].append(w)
    return list(g.values())
def first_unique_char(s):
    c = Counter(s)
    return next((i for i, ch in enumerate(s) if c[ch] == 1), -1)
def top_k_frequent(nums, k): return [x for x, _ in Counter(nums).most_common(k)]
print(is_anagram("listen", "silent"), group_anagrams(["eat", "tea", "tan", "ate", "nat",
"bat"]), first_unique_char("leetcode"), top_k_frequent([1, 1, 1, 2, 2, 3], 2))
```

OUTPUT

```
[0, 1] [1, 2] []
True [['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']] 0 [1, 2]
```

Pattern 2: prefix sums

Precompute running totals so any range sum is O(1).

Python

```python
from itertools import accumulate
sales = [5, 3, 8, 6, 2, 7]
prefix = [0] + list(accumulate(sales))               # prefix[i] = sum of first i items
range_sum = lambda i, j: prefix[j + 1] - prefix[i]    # sum of sales[i..j]
print(prefix, range_sum(1, 3), range_sum(0, 5))

def subarray_sum_equals_k(nums, k):
    """Count contiguous subarrays summing to k. O(n) with prefix sums + a counter."""
    count, running, seen = 0, 0, Counter({0: 1})
    for x in nums:
        running += x
        count += seen[running - k]
        seen[running] += 1
    return count
print(subarray_sum_equals_k([1, 1, 1], 2), subarray_sum_equals_k([1, 2, 3, -3, 3], 3))
```

OUTPUT

```
[0, 5, 8, 16, 22, 24, 31] 17 31
2 5
```

Pattern 3: two pointers

Two indices move toward each other (or in the same direction) through a **sorted** array or string.

Python

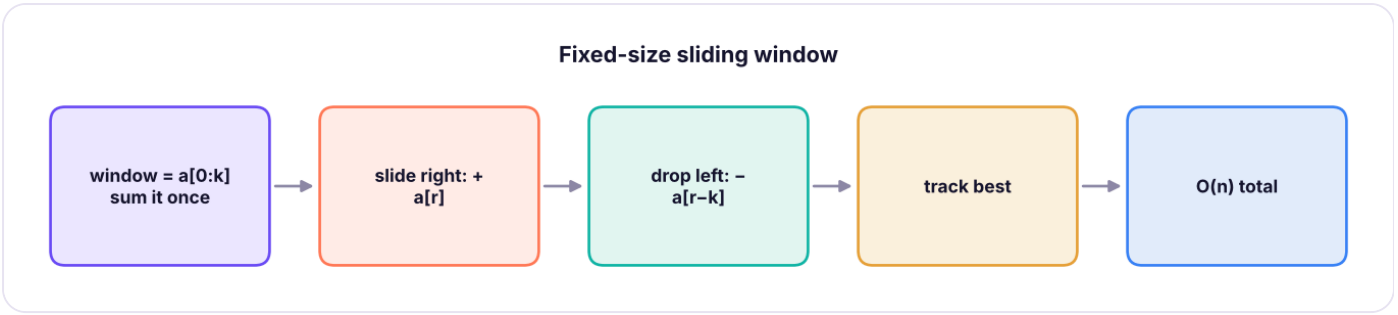
```python
def is_palindrome(s):
    s = [c.lower() for c in s if c.isalnum()]
    i, j = 0, len(s) - 1
    while i < j:
        if s[i] != s[j]: return False
        i += 1; j -= 1
    return True
def pair_with_sum_sorted(a, target):
    i, j = 0, len(a) - 1
    while i < j:
        total = a[i] + a[j]
        if total == target: return a[i], a[j]
        if total < target: i += 1
        else: j -= 1
    return None
def remove_duplicates_sorted(a):              # in place, returns new length
    if not a: return 0
    w = 1
    for r in range(1, len(a)):
        if a[r] != a[w - 1]: a[w] = a[r]; w += 1
    return w
def merge_sorted(a, b):
    i = j = 0; out = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]: out.append(a[i]); i += 1
        else: out.append(b[j]); j += 1
    return out + a[i:] + b[j:]
arr = [1, 1, 2, 3, 3, 3, 4]; n = remove_duplicates_sorted(arr)
print(is_palindrome("A man, a plan, a canal: Panama"), pair_with_sum_sorted([1, 3, 4, 6,
9], 10), arr[:n], merge_sorted([1, 4, 9], [2, 3, 10]))
```

OUTPUT

```
True (1, 9) [1, 2, 3, 4] [1, 2, 3, 4, 9, 10]
```

Pattern 4: sliding window

Keep a window `[left, right]` and slide it, updating a running state instead of recomputing.



A sliding window adds the new element on the right and removes the old one on the left.

Python

```
def max_sum_window(a, k):
    window = best = sum(a[:k])
    for r in range(k, len(a)):
        window += a[r] - a[r - k]
        best = max(best, window)
    return best
def longest_unique_substring(s):
    last, left, best = {}, 0, 0
    for right, ch in enumerate(s):
        if ch in last and last[ch] ⩾ left: left = last[ch] + 1     # shrink past the
repeat
        last[ch] = right; best = max(best, right - left + 1)
    return best
def min_window_at_least(nums, target):          # shortest subarray with sum ⩾ target
(positive numbers)
    left = total = 0; best = float("inf")
    for right, x in enumerate(nums):
        total += x
        while total ⩾ target:
            best = min(best, right - left + 1); total -= nums[left]; left += 1
    return 0 if best == float("inf") else best
print(max_sum_window([2, 1, 5, 1, 3, 2], 3), longest_unique_substring("abcabcbb"),
longest_unique_substring("pwwkew"), min_window_at_least([2, 3, 1, 2, 4, 3], 7))
```

OUTPUT

```
9 3 3 2
```

The same idea appears in SQL window functions and in pandas `rolling`: a 7-day moving average is a sliding window.

Classic array problems

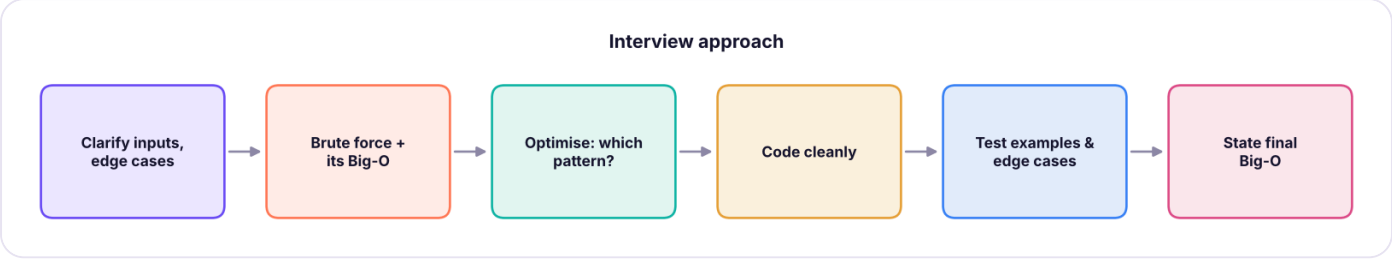
Python

```
def max_profit(prices):                          # best single buy then sell
    lo, best = float("inf"), 0
    for p in prices: lo = min(lo, p); best = max(best, p - lo)
    return best
def max_subarray(nums):                          # Kadane's algorithm
    cur = best = nums[0]
    for x in nums[1:]: cur = max(x, cur + x); best = max(best, cur)
    return best
def product_except_self(nums):
    n = len(nums); out = [1] * n; left = right = 1
    for i in range(n): out[i] = left; left *= nums[i]
    for i in range(n - 1, -1, -1): out[i] *= right; right *= nums[i]
    return out
def move_zeroes(a):
    w = 0
    for x in a:
        if x != 0: a[w] = x; w += 1
    a[w:] = [0] * (len(a) - w); return a
def rotate(a, k): k %= len(a); return a[-k:] + a[:-k]
print(max_profit([7, 1, 5, 3, 6, 4]), max_subarray([-2, 1, -3, 4, -1, 2, 1, -5, 4]),
product_except_self([1, 2, 3, 4]), move_zeroes([0, 1, 0, 3, 12]), rotate([1, 2, 3, 4, 5],
2))
```

OUTPUT

```
5 6 [24, 12, 8, 6] [1, 3, 12, 0, 0] [4, 5, 1, 2, 3]
```

How to answer in an interview



A calm, structured approach scores better than jumping into code.

Patterns to try, in order: **sort? hash map? two pointers? sliding window? prefix sum? stack? binary search? BFS/DFS? DP?** (the next two lessons cover the rest).

Summary

KEY TAKEAWAYS

- Big-O measures growth: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$; drop constants, keep the dominant term.
- `in` on a list is O(n), on a set/dict O(1); inserting at the front of a list is O(n), use `deque`.
- Hash maps/Counters turn searches into lookups (two-sum, anagrams, frequency).
- Prefix sums give O(1) range sums; two pointers work on sorted data and palindromes; sliding windows handle subarray/substring problems in O(n).
- In interviews: clarify, brute force, optimise with a pattern, code, test, and state complexity.

Quick quiz

6 questions · tap an answer to check it

1. What is the time complexity of checking `x in my_set` ?

A O(1) on average

B O(n)

C O(log n)

D $O(n^2)$

Answer: A. Sets are hash tables.

2. Two nested loops over the same list of n items are...

A O(n)

B O(n log n)

C $O(n^2)$

D O(2n)

Answer: C. Nested steps multiply.

3. Which pattern solves "longest substring without repeating characters" in O(n)?

A Binary search

B Sliding window with a dictionary of last positions

C Sorting

D Prefix sums only

Answer: B. Shrink the window past repeated characters.

4. What does the two-sum hash map store?

A Each value seen so far mapped to its index

B All pairs

C The sorted list

D Only the target

Answer: A. Check whether target − x was already seen.

5. Which algorithm finds the maximum subarray sum in O(n)?

A Merge sort

B Kadane's algorithm

C Binary search

D Bubble sort

Answer: B. Keep the best sum ending at each position.

6. What is $O(3n + n^2 + 10)$ simplified?

A $O(3n)$

B $O(n^2)$

C $O(n)$

D $O(10)$

Answer: B. Keep the dominant term.

EXERCISE

Practice set (try before opening the solutions)

(1) Return the indices of all pairs that sum to a target. (2) Check if a string has all unique characters. (3) Find the majority element (appears > n/2 times). (4) Longest run of consecutive 1s after flipping at most one 0 (sliding window). (5) Count customers whose total spend is in the top 10% (prefix sums not needed: think sorting).

Show solution (worked answer)

Python

```python
def all_pairs(nums, t):
    seen, out = defaultdict(list), []
    for i, x in enumerate(nums):
        out += [(j, i) for j in seen[t - x]]; seen[x].append(i)
    return out
def all_unique(s): return len(set(s)) == len(s)
def majority(nums):                       # Boyer-Moore voting, O(1) space
    cand = cnt = 0
    for x in nums:
        if cnt == 0: cand = x
        cnt += 1 if x == cand else -1
    return cand
def longest_ones(a, k=1):
    left = zeros = best = 0
    for right, x in enumerate(a):
        zeros += x == 0
        while zeros > k: zeros -= a[left] == 0; left += 1
        best = max(best, right - left + 1)
    return best
print(all_pairs([1, 5, 3, 3, 7], 6), all_unique("data"), majority([2, 2, 1, 1, 2]),
longest_ones([1, 0, 1, 1, 0, 1]))
```

INTERVIEW CORNER

Questions you may be asked

Q: Find duplicates in a list of 10 million customer IDs. Use a set (or `Counter`) in one pass, O(n) time and O(n) space; in pandas `df.id.duplicated()`; in SQL `GROUP BY id HAVING COUNT(*) > 1`.

Q: Why is sorting O(n log n)? Comparison sorts split the problem in half repeatedly (log n levels) and do O(n) work per level; Python uses Timsort.

Q: Space vs time trade-off example? Two-sum: O(n²) time with O(1) space, or O(n) time with O(n) extra space for a hash map.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.